

Paulos Yibelo - Hacking Research: Why CSP Should be carefully crafted: Twitter XSS & CSP Bypass

Paulos Yibelo - Hacking Research: Why CSP Should be carefully crafted: Twitter XSS & CSP Bypass - xxxxx, Paulos Yibelo - Hacking Research.

- Published: date not stated
- Original: <<http://www.paulosyibelo.com/2017/05/twitter-xss-csp-bypass.html>>
- Current location: <<https://www.evil.blog/2017/05/twitter-xss-csp-bypass.html>>
- Preserved from: <https://www.evil.blog/2017/05/twitter-xss-csp-bypass.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Few months back, I came across an oauth xss accompanied by a nice CSP bypass in Twitter. While creating an application, a developer can set their terms and service URL for their app, which Twitter configured to be: `* ([https?:])\w+ *`

Unfortunately the regexp is missing a `^` char in the start making malicious URLs like `data:CONTENT#https://...` work -- so we got HTML Injection, but almost useless for a practical attack because of the CSP rules. After checking the header, I noticed there are multiple CSP misconfigurations in the script-src and object-src blocks, making it possible to bypass CSP in twitter.com. The CSP Rule looks like:

```
script-src https://connect.facebook.net https://cm.g.doubleclick.net https://ssl.google-  
analytics.com https://graph.facebook.com https://twitter.com 'unsafe-eval' 'unsafe-inline'  
https://*.twimg.com https://api.twitter.com https://analytics.twitter.com  
https://publish.twitter.com https://ton.twitter.com https://syndication.twitter.com  
https://www.google.com;frame-ancestors 'self';object-src https://twitter.com  
https://pbs.twimg.com; default-src 'self';...
```

Looking at this, the object-src and the script-src blocks got my immediate attention.

After some research, I saw one of the trusted domains (cdn.syndication.twimg.com aka syndication.twitter.com) hosts JSONP endpoints.

Originally I thought, by exploiting the object-src block (<https://pbs.twimg.com>) -- one can upload a Flash file (as picture/video extension with few bytes header) to Twitter CDN -- refer it to as an embedded Object to gain code execution. However, because of character limitation, the payload I was trying to make was too long and being cut off, so this method wasn't practical as we were

working on a limited payload space. At this point, I stuck to the JSONP bypass for the script-src blocks and started playing with multiple parameters until I found a shorter version, when injected generating an alert in twitter.com.

```
http://syndication.twitter.com/widgets/timelines/246079887021051904?  
dnt=true&domain=twitter.com&lang=en&callback=alert
```

The above JSONP response from syndication.twitter.com comes back with a Content-Disposition header forcing a download. However, browsers like Chrome still execute the returned file even when returned as an attachment. At this point, this misconfiguration added with the 'unsafe-inline' CSP block -- meant we are able to execute code.

By setting the Terms & Services URL of an App to

```
data:text/html,<script src="https://syndication.twitter.com/widgets/timelines/246079887021051904?callback=alert"></script>
```

A developer will be able to pop-up an alert.

POC

After some digging I noticed ssl.google-analytics.com, www.google.com and even graph.facebook.com host JSONP endpoints -- which I wrote to twitter over email -- but will not be fixed anytime soon because it may break the sites usage and call to these sites and performance.

Edit: Ben Hayak [mentioned](#) we can use same origin method execution (SOME) attack to manipulate the page as we like:

```
https://syndication.twitter.com/widgets/timelines/246079887021051904?  
callback=document.body.firstChild.Reference.submit -- as used by my Instagram XSS.
```

I hope it was a fun read, :) --

Share This Story

**Tags:

[Newer Post](#) [Older Post](#)